

基于卷积神经网络的表情识别案例

一、任务描述

本任务是谷歌Kaggle人工智能大赛项目，表情识别是一个比较有挑战性的任务，数据集采用fer2013（facial expression recognition 2013），训练集包括28709张图片、测试集和验证集各包括3589张图片，均为(48, 48)的灰度图像。要求搭建卷积神经网络训练模型并保存，然后识别自定义的人脸图片表情。



本任务的主要工作内容：

- 1) 加载数据，分离特征和标签并可视化查看几组数据
- 2) 搭建卷积神经网络，训练并保存
- 3) 加载保存的模型，识别自定义图片

二、任务目标

- 1、熟练掌握卷积神经网络的搭建
- 2、熟练掌握模型的编译及训练方法
- 3、熟练掌握模型评估方法
- 4、熟练掌握模型的保存、加载
- 5、掌握使用模型预测自定义图片的方法

三、任务环境

操作系统：Ubuntu18.04/Windows10

软件环境：Anaconda3 2019(Python3.7)、Tensorflow2.1.0

四、任务分析

Fec2013表情数据集中包括7种常见的表情，分别用数字0~6来表示，如下：



其中，训练集包括28709张（48, 48）的灰度模式表情图片，用来训练卷积神经网络模型，预测自定义图片的表情。



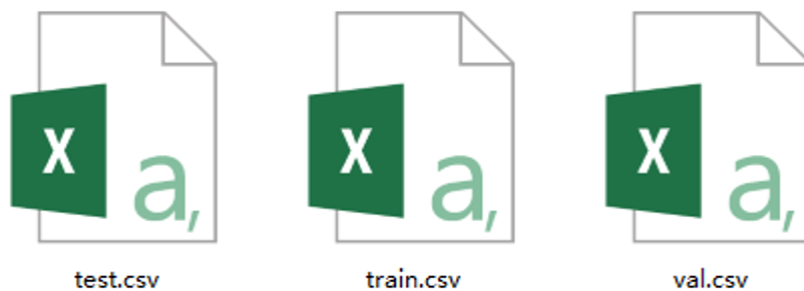
本实验的主要实施环节为：

- 1) 加载数据，分离特征和标签
- 2) 可视化查看几组数据
- 3) 搭建卷积神经网络，训练并保存
- 4) 加载保存的模型，识别自定义图片

五、任务步骤

步骤一、加载和处理数据

这里，我们将训练集、测试集和验证集三个csv文件放置在当前代码目录下的 `data/fer2013` 目录中。



每个Csv文件中，都包括图片的标签（emotion）和像素值（pixels），并且像素值是用空格隔开的，所以需要拆分。

	emotion	图片像素值，用空格隔开，需分割	pixels
0	标	0	70 80 82 72 58 58 60 63 54 58 60 48 89 115 121...
1	签	0	151 150 147 155 148 133 111 140 170 174 182 15...
2		2	231 212 156 164 174 138 161 173 182 200 106 38...
3		4	24 32 36 30 32 23 19 20 30 41 21 22 32 34 21 1...
4		6	4 0 0 0 0 0 0 0 0 0 0 0 3 15 23 28 48 50 58 84...
...	...		
28704	2		84 85 85 85 85 85 85 85 86 86 86 87 86 86 91 9...
28705	0		114 112 113 113 111 111 112 113 115 113 114 11...
28706	4		74 81 87 89 95 100 98 93 105 120 127 133 146 1...
28707	0		222 227 203 90 86 90 84 77 94 87 99 119 134 14...
28708	4		195 199 205 206 205 203 206 209 208 210 212 21...

train.csv

28709 rows × 2 columns

28709张图片，两列（标签和特征值）

- 首先，使用pandas加载csv文件，然后定义一个通用函数来拆分其中的特征值和标签，之后调用该函数，获得拆分好的训练集、测试集和验证集。

```
import numpy as np
import pandas as pd

# 使用pandas读取数据集csv文件
train_data = pd.read_csv('data/fer2013/train.csv') # 训练集
test_data = pd.read_csv('data/fer2013/test.csv') # 测试集
val_data = pd.read_csv('data/fer2013/val.csv') # 验证集

# 定义一个函数，用来分离数据集中的特征和标签
def split_data(data):
    x, y = [], []
    pixels = data['pixels'] # 像素值列
    emotions = data['emotion'] # 标签列
    for pixel, emotion in zip(pixels, emotions):
        pixel = np.array(pixel.split(' '), 'float32') # 像素值用空格分割
        x.append(pixel)
        y.append(emotion)

    x = np.array(x) # 转成numpy数组
    y = np.array(y)
    x = x.reshape(-1, 48, 48, 1) # 数据转为4D张量
    return x, y

# 调用函数，获得训练集、测试集和验证集
x_train, y_train = split_data(train_data)
x_test, y_test = split_data(test_data)
x_val, y_val = split_data(val_data)

print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)
print(x_val.shape, y_val.shape)
```

步骤二、查看训练集中的前几张照片

使用Matplotlib输出查看训练集中的前4张图片（注意imshow方法——根据像素值矩阵输出图片）。

```
import matplotlib.pyplot as plt

# 查看前4张图片
for i in range(4):
    plt.subplot(1, 4, i + 1) # 画子图(1行4列)
    plt.gray()
    plt.imshow(X_train[i].reshape([48, 48])) # 显示当前图像

plt.show()
```

步骤三、搭建卷积神经网络，训练并保存

示例代码中，我们使用了3个卷积层和2个全连接层实现。需要注意的是，在具体任务中，神经网络的层数、每层神经元的个数（输出层除外）一般会结合硬件能力及模型的性能来调整。

注意，输出层共7个神经元，对应7种表情，激活函数使用softmax。softmax是一个多分类激活函数，一般用在多分类网络的输出层。

```
from tensorflow.keras import layers, models

batch_size = 64 # 每批数据量（根据电脑硬件情况设置，过大会耗尽资源）
epochs = 2 # 训练迭代次数

model = models.Sequential() # 创建序贯模型

# 第一层卷积
model.add(layers.Conv2D(32, (3, 3), input_shape=(48, 48, 1), activation='relu'))
model.add(layers.Conv2D(32, (3, 3), activation='relu'))
model.add(layers.MaxPool2D())

# 第二层卷积
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPool2D())

# 第三层卷积
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.Conv2D(128, (3, 3), activation='relu'))
model.add(layers.MaxPool2D())

model.add(layers.Flatten()) # 压平，交给后面的Dense层

# 全连接层
model.add(layers.Dense(64, activation = 'relu'))
model.add(layers.Dense(7, activation = 'softmax'))

# 编译训练
model.compile(loss = 'sparse_categorical_crossentropy', optimizer = 'rmsprop',
metrics = ['acc'])
model.fit(X_train, y_train, batch_size = batch_size, epochs = epochs)
```

```
# 评估模型
score = model.evaluate(X_test, y_test)
print('Test acc:', score[1])

# 保存模型
# model.save('model/fer.h5')
```

- 执行完毕后，会在指定目录下生成模型文件。

名称	修改日期	类型	大小
 fer.h5	2020/5/20 13:23	H5 文件	2,569 KB

- 这样以后使用模型时，不需要再进行训练，直接加载即可（如下）。

```
from tensorflow.keras.models import load_model

model = load_model('model/fer.h5') # 加载模型文件
model.summary() # 查看模型信息
```

步骤四、使用模型识别指定图片的表情



22.png



4.png



13.png

用训练好的模型来识别指定的图片，关键是需要将图片处理成与训练集数据相同的格式，包括尺寸、颜色模型、角度等。

```
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
import numpy as np

# 定义一个函数，显示图片并识别表情
def predict_show(src):
    plt.figure(figsize=(6, 3)) # 图片大小
    plt.subplot(1, 2, 1)
    img = image.load_img(src) # 读入并显示原图
    plt.imshow(img)

    # 处理图片、调用模型识别表情
```

```

labels = ['angry', 'disgust', 'fear', 'happy', 'sad', 'surprise', 'natural']
# 全部表情
img = image.load_img(src, grayscale = True, target_size= (48, 48)) # 灰化、调整大小
x = image.img_to_array(img) # 转数组
x = x.reshape(-1, 48, 48, 1) # 转成模型所需张量格式
result = model.predict(x) # 调用模型进行识别

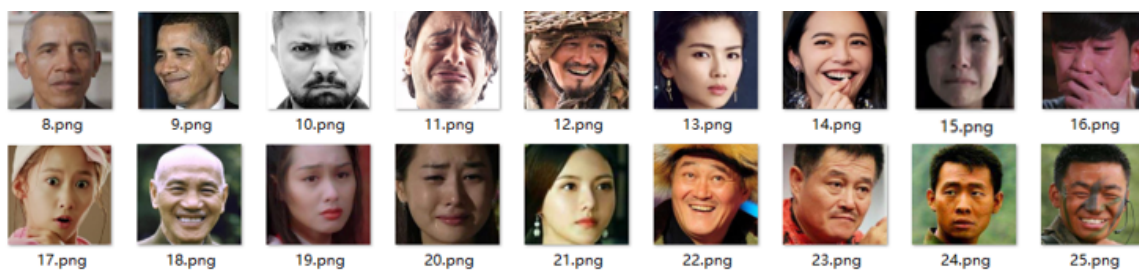
# 显示识别结果
emotion = labels[np.argmax(result)] # 预测结果
plt.subplot(1, 2, 2)
plt.gray()
plt.imshow(x.reshape(48, 48)) # 显示图片
plt.title(emotion, fontsize=25) # 显示表情标签
plt.show()

# 预测指定图片表情
src = 'data/20.png'
predict_show(src) # 调用函数完成

```



步骤五、识别指定目录下所有图片的表情



```

import os

# 识别目录下所有图片
path = 'data'
files = os.listdir(path) # 读取目录
for file in files:
    if not os.path.isdir(file) and file.endswith('.png'):
        predict_show(path+'/'+file) # 调用函数识别并显示

```

识别结果 (部分)

